

# CS 284: Programming Assignment 2

Due: 2026-03-09

*This is a paired assignment with Homework 2. It is recommended that you read through both assignments before beginning, and that you work on them in parallel.*

In class, we have seen both that both a `LinkedList` and an `ArrayList` can take up to linear time on certain operations. This is undesirable, and makes certain operations inefficient to perform using them.

Instead, we would like to find a more efficient implementation. Or at least one that can handle all operations somewhat efficiently, even if it isn't perfect at any particular one.

And in this assignment, that is exactly what you will implement.

## Introduction: On Triangular Lists

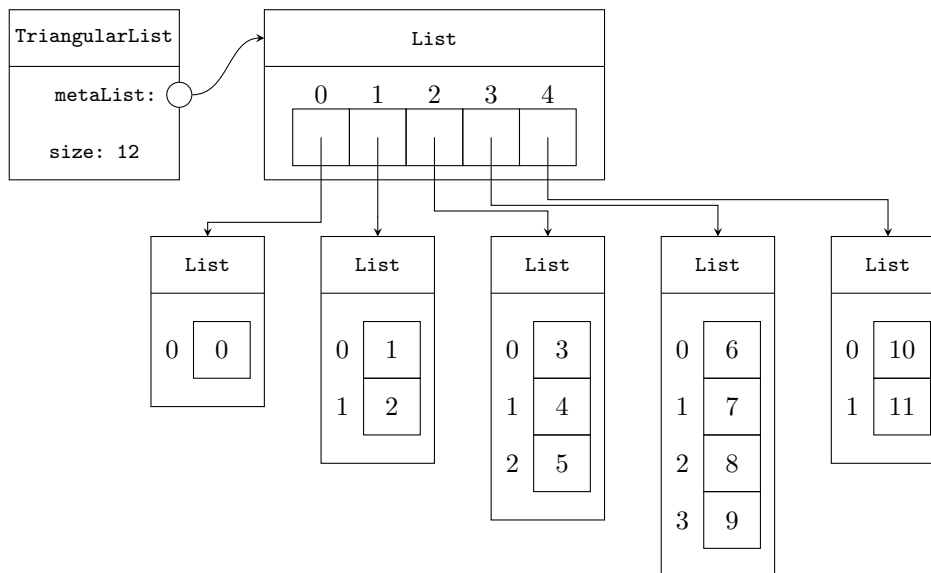
In this assignment, you will implement the `TriangularList` class. This is an implementation of a `List` which itself uses a `List` of `Lists`.

More specifically, the `TriangularList` stores two variables: `size`, which contains the number of elements in the `TriangularList`; and `metaList`, which is an object of type `List<List<E>>`, where `E` is a generic type denoting the type of the contents of the `TriangularList`.

The elements in the `TriangularList` are then stored in the inner lists stored in `metaList`. The order of these elements (and thus their indices) is that we first traverse the entire first inner list, then the entire second inner list, and so on.

The catch is in how these inner lists are structured: The first `List` in the `metaList` stores only one element. The next stores two, then three, and so on. In other words, each inner list is one element longer than the previous one, with the exception of the last element in the chain (which is allowed to be incomplete).

So for example, the following is a graphic representation of a `TriangularList` containing the nonnegative integers up to 11. Each index also contains its own value (so the 7 is in index 7 in the `TriangularList`, for example):



Note, however, that those inner `List`s are of an unspecified type: Is the `metaList` an `ArrayList` of `ArrayList`s? Is it a `LinkedList` of `LinkedList`s? Is it some mixture of the above?

In this assignment, you will thus not only implement a `TriangularList`, but figure out which combination of types gets the best performance, and analyse it as well.

### A Note on Access Control

Note that traditionally, your `TriangularList` would likely store its `metaList` as a `private` field, to separate its specification from its implementation.

However, we need to be able to access the internals of your class to be able to run our tests. As such, we require that the fields we provide you must be marked `public`, so that our grading scripts can access them.

You may, however, mark any helper functions your create as `private`.

### A Note on Generics

Note that we want our `TriangularList` to be usable with any type, not restricted to just storing `Strings` or integers. As such, the class definition is given in such a way that it contains one Generic parameter, allowing it to contain any type.

### A Note on Java's List Classes

For this homework, we are using slightly different names for the `List` methods than Java uses. However, we also expect you to use Java's built-in `List` implementations for your inner lists.

To get the full `List` API you should check Java's documentation (<https://docs.oracle.com/javase/8/docs/api/java/util/List.html>), but we provide you with a simplified description below:

| Our Method Name     | Java's Method Name  | Description                                                                                         |
|---------------------|---------------------|-----------------------------------------------------------------------------------------------------|
| <code>get</code>    | <code>get</code>    | Retrieves the item at a given index in the <code>List</code> .                                      |
| <code>set</code>    | <code>set</code>    | Updates a given index in the <code>List</code> to be the provided value.                            |
| <code>insert</code> | <code>add</code>    | Inserts the element at a given index in the <code>List</code> , shifting over the rest of the list. |
| <code>remove</code> | <code>remove</code> | Removes the element at a given index in the <code>List</code> .                                     |
| <code>size</code>   | <code>size</code>   | Returns the number of elements in the <code>List</code> .                                           |

For good measure, we also mention that Java's `List` API specifies `First` and `Last` variations of a lot of the above methods. For example, `l1.getLast()` would return the last element of the `List` `l1`, while `l1.addFirst(x)` would insert `x` at the start of `l1`.

Additionally, both of Java's `List` implementations are quite a bit more efficient than the ones we saw in lecture. Notably, the `LinkedList` class actually stores a doubly-linked list (a topic we will cover later in the semester), where each operation takes time proportional to its distance from the endpoints of the `List`. In other words, appending an element and prepending an element both would take  $O(1)$  time, for example.

Meanwhile, the `ArrayList` class has the ability to allocate more space than the list needs. This extra space can be used to implement appending efficiently in most cases, except for when the list is currently full. It technically runs in what is called "amortized constant" time, but this is a topic which we will not be covering in this class. For our purposes, the worst-case complexity is still  $O(n)$ , as we sometimes do need to copy over the entire list.

## Implementation Requirements

Your task is to implement the `TriangularList<E>` API. In particular, your `TriangularList` should implement the following methods:

**default Constructor:** Your `TriangularList` should have a default zero-argument constructor. In other words, I should be able to call `new TriangularList<>()` to create a new (empty) `TriangularList`.

**get:** `get(idx)` should return the value in the `List` at index `idx`.

Note that like all methods in this class, this is generic, and should be able to accept any type if used with a `TriangularList` of that type.

**set:** `set(idx,value)` should update the value at index `idx` of the `List` to be `value`.

**insert:** `insert(idx,value)` should insert `value` into the list such that it ends up at index `idx`, with values at or after `idx` being shifted right.

**remove:** `remove(idx)` should remove the value at the index `idx` from the list. This should cause the list to shrink, with elements after `idx` moving up.

**size:** `size()` should return the number of elements in the list.

**toString:** (*OPTIONAL*) `toString()` should return a `String` representation of the `List` it is called on.

You may find this last function useful for using the Java debugger.

**Don't forget to include input validation on all your functions!** You should throw a `RuntimeException` on any function call that should result in an error.

## Suggested Implementation Order

Below, we give a suggested approach for how to tackle the above tasks, as well as a few hints on how to do it.

- Start with the constructor. How should the fields be instantiated so that the initial `TriangularList` is empty?
- Next, you might find it useful to implement some helper methods. Given an index `idx`, which of the inner lists is that element located in? And where in that inner list?  
This method can be implemented in one of two ways: Either by performing a bit of math, or by taking calculating it the brute-force way. You can choose to use either.

- Implement `get` and `set` next.

If you have the above helper functions, this should be quite easy.

- Implement the tail end case of `insert`.

In other words, implement what happens if you were to `insert` only into the last inner list, and you did not need to create a new inner list.

- Implement the general case of `insert`.

This is the case that you do not need to create an inner list. You should already have code for inserting into the last inner list. Then, note that inserting into any other inner list results in that inner list having one element too many. Think about how you can fix this to maintain the invariant that the inner lists are in ascending size.

- Implement the full version of `insert`, by handling the case where a new inner list has to be created.

- Implement the general case of `remove`.

This should be done symmetrically to `insert`, and you will find a lot of similarities if your code is set up properly.

- Modify your previous methods to maintain the `size` field correctly.

- Implement `size`.
- Add input validation to all your methods.

Remember to test as you go, making sure that your `TriangularList` behaves as anticipated.

Additionally, remember that you are always allowed to write helper functions, which might be particularly useful when certain operations are repeated often.

## Submission Instructions

For this homework, you must submit the following files, with exactly the following names:

- `TriangularList.java`

Note that `TriangularList` must be located in the `pa2` package, or we will be unable to compile it otherwise.

Submissions must be handled through Gradescope (accessible through the homework page on Canvas). Grades will be released through Gradescope once the homework is graded.

## Rubric

| Category                                            | Description                                                     | Points |
|-----------------------------------------------------|-----------------------------------------------------------------|--------|
| Implementation: <code>get</code> & <code>set</code> | Correct implementation of <code>get</code> and <code>set</code> | 20     |
| Implementation: <code>insert</code>                 | Correct implementation of <code>insert</code>                   | 25     |
| Implementation: <code>remove</code>                 | Correct implementation of <code>remove</code>                   | 25     |
| Implementation: <code>size</code>                   | Correct implementation of <code>size</code>                     | 10     |
| Input Validation                                    | Correctly handling invalid inputs in the above methods          | 20     |
| <b>Total</b>                                        |                                                                 | 100    |